

NodeFlow — estado, investigación y hoja de ruta

Qué construimos, qué averiguamos, dónde estamos exactamente y qué falta. Todo lo que dice «medido» en este informe se volvió a correr el 19/09/2026 sobre el repositorio real; lo que no se pudo verificar está marcado como tal.

TOMAS.WAV · NodeFlow 0.3.7 · repositorio `github.com/NeonEden/NodeFlow` · rama `main` en `d11f5d3`

0 · Cómo leer esto

Cuatro marcas, siempre visibles:

- **medido** lo corrí en esta máquina o lo leí del repositorio en esta sesión.
- **construido** existe, está en `main` y pasa CI.
- **pendiente** está identificado, con esfuerzo estimado, y no está hecho.
- **descartado** se evaluó y no conviene hacerlo ahora — con el motivo.

Los esfuerzos son estimaciones, no mediciones. Las afirmaciones que vinieron de investigación delegada se conservaron sólo cuando el sondeo contra la fuente las confirmó; las que no, están anotadas con su prueba.

1 · Resumen ejecutivo

292 TESTS EN VERDE	4 PR MERGEADOS HOY	2 INFORMES DE INVESTIGACIÓN	1 MIGRACIÓN EN CURSO
------------------------------	------------------------------	---------------------------------------	--------------------------------

La app funciona y se instala. El instalador NSIS sale con marca, accesos directos, entrada en «Aplicaciones instaladas» y desinstalador. Los 292 tests pasan (282 en Rust, 10 en el frontend), la CI está verde y el demo web está publicado en Vercel. La ventana ya tiene una política de seguridad de contenido (CSP) propia, y el estado se escribe de forma atómica.

La investigación cambió dos decisiones de fondo. La primera: la librería central del lienzo (`reactflow` v11) está en la rama muerta del proyecto; la viva es `@xyflow/react` 12. La segunda: las apps de agente grandes que tenés instaladas (Claude Desktop, Qwen, Antigravity, Copilot) **no exponen línea de comandos**, así que no sirven como trabajadores llamables — lo que sí sirve es `opencode`, y ese hallazgo es el que habilitó todo el flujo de delegación que armamos.

Lo que falta no es código, son tres cosas concretas. (1) Un certificado de firma *real*: el auto-firmado que pusimos prueba la tubería, pero no quita el aviso de SmartScreen. (2) La migración a React Flow 12, que ya compila y corre pero todavía no está commiteada ni rebasada sobre `main`. (3) La clave de API propia del usuario, que hoy no está resuelta — ver §6.3.

El deadline que manda

El **30/09** cierran el video demo del hackathon y los 30 prospectos de outreach. Todo lo que sigue en este informe es el *medio*; el *fin* es el ciclo de ingresos. El ecosistema de agentes es infraestructura interna: si se come la semana, el costo de oportunidad es todo el ciclo.

2 · Dónde estamos, medido

Pieza	Estado	Evidencia de esta sesión
Repositorio <code>main</code>	medido	<code>d11f5d3</code> · 203 commits · PR #9, #10, #11, #12 mergeados
Tests backend (Rust)	medido	<code>cargo test</code> → 282 passed; 0 failed (0.94 s)
Tests frontend (vitest)	medido	<code>npx vitest run</code> → 2 archivos, 10 passed (<code>layout.test.ts</code> , <code>obsidianExport.test.ts</code>)
Chequeo de tipos	medido	<code>tsc --noEmit</code> limpio · CI verde en los 4 runs de merge
CSP de la ventana	construido	<code>tauri.conf.json</code> con <code>csp</code> + <code>devCsp</code> ; verificada en el binario de release sin violaciones
Estado atómico	construido	un solo camino de escritura (<code>estado.rs</code> / <code>vault.rs</code>), con sufijo temporal <code>tmp-nf</code>
Instalador NSIS	medido	<code>NodeFlow_0.3.7_x64-setup.exe</code> · 5.057.552 bytes · firmado
Instalador MSI	medido	<code>NodeFlow_0.3.7_x64_en-US.msi</code> · 7.303.168 bytes · sin firmar
Firma Authenticode	parcial	certificado auto-firmado <code>CN=NodeFlow Test</code> , válido hasta 19/09/2027 · <code>signtool verify</code> → «cadena termina en una raíz no confiable» (esperado)
Demo web	construido	Vercel con <code>rootDirectory=demo</code> y el bundle versionado en <code>demo/public/</code>
Migración React Flow 12	en curso	<code>worktree nodeflow-reactflow12</code> · <code>@xyflow/react ^12.11.6</code> · 17 archivos, +202/-488 · compila · falta commit y rebase
Worktrees	medido	<code>nf-csp-tests</code> , <code>nf-estado-atomico</code> , <code>nf-warnings</code> (mergeados), <code>nodeflow-reactflow12</code> (activo)

La máquina, para contexto

AMD Ryzen 5 5600G (6 núcleos / 12 hilos) · **15,8 GB de RAM** · AMD Radeon RX 6700 XT. Node v22.23.2 · npm 12.0.2 · cargo/rustc 1.98.1.

Los 15,8 GB importan: Ollama + Docker + varios workers + el navegador no entran a la vez. El paralelismo tiene techo físico — conviene cola explícita, no «lanzar todo».

3 · Qué construimos en esta sesión

3.1 · El instalador que se comporta como un programa

Antes la app se instalaba «a medias»: sin ícono, sin entrada en el menú de inicio, sin aparecer en «Aplicaciones instaladas». Ahora el paquete NSIS tiene branding (encabezado propio, íconos de instalador y desinstalador), crea los accesos directos en escritorio y menú de inicio, registra el desinstalador y renombra el binario a `NodeFlow.exe`. El viaje de ida y vuelta (instalar → desinstalar) se probó completo.

3.2 · Movimiento 1: cerrar los dos huecos que ya eran deuda

CSP. La ventana tenía `csp: null` — cualquier script que entrara por error podía correr. Ahora hay una política explícita que declara de dónde puede venir cada cosa: scripts y estilos propios, imágenes también de DiceBear y Unsplash, medios locales, fuentes locales, y conexiones únicamente al backend local (`127.0.0.1`), a AssemblyAI por WebSocket y a DeepSeek. `object-src 'none'` y `base-uri 'self'` cierran las dos vías clásicas de inyección. Hay una variante `devCsp` más permisiva sólo para el servidor de desarrollo. Se verificó ejecutando el binario de release: la política se inyecta y el log no registra violaciones al cargar la bóveda ni las voces locales.

Vitest y los primeros tests. El frontend no tenía *ninguna* prueba: cualquier refactor era a ciegas. Se montó vitest y se escribieron los primeros tests sobre dos funciones puras — el layout y la exportación a Obsidian. Son 10 tests, pero lo importante no es el número: es que ahora existe la **red**. Sin esto, la migración de React Flow que viene después habría sido un salto al vacío.

3.3 · El estado se escribe en un solo lugar, y de forma atómica

Antes el estado se guardaba desde varios puntos y con escrituras directas: un corte de luz o un cierre forzado a mitad de escritura podía dejar un JSON truncado, y con él la bóveda ilegible. Ahora hay un único camino de escritura que escribe a un archivo temporal y lo mueve al destino — la operación es atómica, o pasa entera o no pasa. Lo produjo un worker delegado y se verificó con la suite completa.

3.4 · El crate sin warnings de código

El build de Rust arrastraba código muerto y avisos. Se limpiaron; el árbitro (`cargo test`) confirmó que no se rompió nada. También es el PR que estrenó el flujo de delegación de punta a punta.

Precisión, medida al cerrar este informe: el PR #9 limpió el crate `app`, pero quedaba un warning real sin tocar — una variable declarada y nunca usada en un test del curador (`curador.rs:539`). Se corrigió usando el fixture que ya estaba declarado (`aristas.len()`) en vez del número escrito a mano, con lo que además el test quedó autoconsistente. Queda un aviso de `linker_messages` que emite el **enlazador de MSVC**, no el código: es ruido del toolchain de Rust 1.98, no deuda del proyecto.

3.5 · El demo web vuelve a estar en pie

Vercel estaba rojo porque el bundle no existía en el repositorio. Se corrigió configurando el directorio raíz del proyecto en `demo` y versionando `demo/public/` (que estaba en el `.gitignore`). El deployment quedó `Ready`.
Nota: esta es la pieza que va a servir de escaparate para los 30 prospectos.

3.6 · Movimiento 2: la migración a React Flow 12 en curso

El informe de la pila reveló que `reactflow` v11 — la librería sobre la que está construido todo el lienzo — está en la **rama muerta** del proyecto. La rama viva es `@xyflow/react` 12. Se hizo la migración en un worktree aparte

(`nodeflow-reactflow12`), con los árbitros corriendo: 17 archivos tocados, +202/−488 líneas (el saldo negativo es porque la v12 absorbe código que antes había que escribir a mano).

Diferencias concretas que aparecieron:

- El paquete cambia de nombre: `reactflow` → `@xyflow/react`.
- La exportación por defecto desaparece: `import ReactFlow, {...}` pasa a ser `import { ReactFlow, ... }` — el componente ahora es una exportación con nombre.
- La hoja de estilos se mueve a `@xyflow/react/dist/style.css`.
- Y el detalle que costó más tiempo: **no existe ninguna versión 12 publicada como `reactflow`**. El `npm install` fallaba con `No matching version found for reactflow@^12.0.0` porque -ese paquete nunca pasó de la 11. La 12 es otro paquete. Eso no estaba en ningún informe: se descubrió chocando contra el error.

El resultado compila (`vite build` genera los assets) y la app corre en modo desarrollo con el lienzo funcionando. **Lo que falta:** commitear, rebasar sobre `main` (el worktree quedó parado en `637c328`, antes de los 4 merges) y abrir el PR con los árbitros en verde.

3.7 · El flujo de delegación: cómo se fabricó todo lo anterior

Esta es la pieza que no se ve pero que cambió la forma de trabajar. Se armó un contrato de repositorio (`AGENTS.md`) que declara comandos canónicos, convenciones y lo que no se toca; un script de delegación (`scripts/delegar.sh`) que prepara un worktree, calienta el build, enlaza `node_modules` y lanza al worker; y **árbitros** — la suite de tests y el chequeo de tipos — que deciden si el trabajo del worker se acepta o se descarta.

La regla es simple y es la que evita el desastre: **el worker propone, los tests deciden**. Los PRs #9 y #11 salieron de workers delegados; ninguno se mergeó sin que la suite pasara entera.

Dos cicatrices que valen

- El **contrato tiene que estar commiteado antes de crear el worktree**. Si no, el worker lee una versión vieja del repo y produce trabajo que no encaja.
- Un worktree «frío» (sin `target/`) hace que el worker devuelva una edición vacía: parece que falló cuando en realidad no llegó a compilar. Hay que calentarlo antes.
- Y la lección más cara, de antes: **un bot con escritura en el repo puede romper el estado de la app** (un reinicio dejó el lienzo en blanco). De ahí la regla de worktree propio, alcance declarado y revisión por tests antes de mergear.

4 · Las investigaciones

Se produjeron dos informes, cada uno sondeado contra la fuente autoritativa (npm, crates.io, la API de GitHub y el propio repositorio) el mismo día. Están en `docs/informes/`.

4.1 · Ecosistema de orquestación de agentes

14 páginas · `NodeFlow-ecosistema-agentes-2026-09-19.pdf`

Relevó qué hay instalado en esta máquina, qué protocolos son estándar de verdad, qué agentes-CLI se pueden llamar, y qué frameworks de orquestación siguen vivos. Termina con teorías de continuación ordenadas por

costo: lo que multiplica ya (Nivel A), lo que abre techo (B) y las apuestas grandes (C).

4.2 · La pila de NodeFlow

8 páginas · `NodeFlow-pila-2026-09-19.pdf`

Qué capacidades de Tauri ya están pagadas y sin usar, qué plugins son realmente oficiales, qué le falta al lienzo comparado con un editor de nodos típico, y qué del ecosistema de Rust conviene. Cierra con «qué haría yo, en este orden» — que es de donde salieron los tres movimientos que ejecutamos hoy.

4.3 · Cómo se verificó

Se corrió un fan-out de tres subagentes para el relevamiento inicial, y **todo lo que devolvieron se sondeó después contra la fuente**. Eso fue lo que detectó los errores: la investigación delegada afirmaba que `tauri-plugin-frameless-window` era oficial (no lo es: es de un particular, con 195 descargas en total), que `sled` iba por la 1.0.0-alpha (la última publicada es 0.34.7, de octubre de 2024), y citaba como estables dos release candidates. Nada de eso entró al informe como hecho. **La regla que quedó: el delegado investiga, el sondeo decide.**

Y el hallazgo que no era de nadie: Kùzu

Los informes y el contrato de agentes hablaban de «grafo Kùzu». Al ir a verificar el código, resultó que **NodeFlow no usa Kùzu**: el grafo es una implementación propia en JSON, en `grafo.rs`, con `serde_json`. La dependencia nunca existió. Se corrigió la documentación y el contrato. Si no se hubiera sondeado, hoy habría decisiones tomadas sobre una base que no existe.

5 · Los hallazgos que cambian decisiones

Hallazgo	Consecuencia
React Flow v11 está en la rama muerta. La viva es <code>@xyflow/react 12</code> .	Era el riesgo estructural más grande: toda la app depende de esa librería. La migración dejó de ser cosmética y pasó a ser la tarea de fondo.
No existe <code>reactflow@12</code>. El nombre del paquete cambió.	Explica por qué la migración no era un bump de versión. Cualquier plan que la tratara como tal estaba mal.
Claude Desktop, Qwen, Antigravity y Copilot no tienen CLI en el PATH.	No se pueden usar como trabajadores. Cualquier arquitectura que las trate como workers necesita un puente que hoy no existe. Su rol real es <i>cliente</i> con humano en el medio.
<code>opencode 1.18.18</code> es un servidor de agentes (<code>run</code> , <code>serve</code> , <code>acp</code>).	Fue el hallazgo que habilitó todo: hoy es el worker del flujo de delegación.
GitHub Models está retirado (30/07/2026). El endpoint responde 410 Gone.	Cualquier rol apoyado en esa API está muerto. No se instala su CLI.
La cuota de búsqueda de Gemini se agota antes que la del modelo. Con herramienta de búsqueda, todos los modelos devolvieron 429; sin ella, el mismo modelo contestó 200.	El diseño de investigación por fases tiene que declarar cuota y plan B por rol. Ya está implementado el fallback Gemini → Tavily → Hermes.
Un agente-CLI como buscador es lento: una búsqueda delegada pasó los 300 s y se cortó.	Sirve como worker de código, no como buscador en camino caliente.
MCP es el único estándar con adopción real (9.254 ★, actividad diaria).	El servidor MCP propio (<code>nodeflow_mcp.py</code>) ya montado es la inversión que se multiplica: sirve a todos los clientes a la vez.
El formato <code>AGENTS.md</code> tiene 24.471 ★ y lo leen todos los agentes.	Es el contrato más barato del ecosistema: se escribe una vez y trabaja para cualquier worker que entre al repo.
El ecosistema de agentes es medio, no fin.	Lo que se vende es el resultado (visuales, automatizaciones, NodeFlow como demostración de criterio). Si el ecosistema se vuelve el proyecto, el imán deja de atraer.

6 · Lo que falta

6.1 · Bloqueantes reales

Qué	Estado	Qué hace falta	Esfuerzo
Firma Authenticode real	parcial	El auto-firmado prueba que la tubería funciona, pero <code>signtool verify</code> da «raíz no confiable» — o sea, no quita SmartScreen . La firma que sí lo quita es un certificado OV/EV o la firma en la nube (Azure Trusted Signing, que con el crédito de Azure ya disponible es la vía barata). Después va <code>bundle.windows.signCommand</code> en <code>tauri.conf.json</code> . El MSI además quedó sin firmar.	Un rato de configuración + la validación del certificado
Migración a React Flow 12	en curso	Commit, rebase sobre <code>main</code> (el worktree quedó en <code>637c328</code>), PR con los árbitros. El riesgo honesto: los tests de layout cubren las funciones puras, no el render del lienzo — hay que probar a mano arrastrar, conectar y el minimapa.	Media tarde
Publicar el instalador firmado	pendiente	Release <code>v0.3.8</code> en GitHub con NSIS y MSI. Hoy hay release <code>v0.3.7</code> publicada, pero es anterior a la CSP, los tests y el estado atómico.	Una hora

6.2 · Pendientes elegidos una tarde corta

El Movimiento 3 del informe: pulir como programa de escritorio. Son cinco cosas que juntas cambian la sensación de «app de verdad»:

- **Bandeja del sistema + atajo global + window-state** (recordar tamaño y posición) — los tres juntos, y los tres son plugins oficiales de Tauri.
- **Soltar archivos sobre el lienzo** — alrededor de una hora, y es una función que ningún competidor local tiene.
- **Paleta de comandos** con `cmdk` + `fuse.js`.

Extra gratis: efectos de ventana (Mica) y menú nativo, que ya vienen en el core de Tauri sin plugin.

6.3 · La clave de API propia: no resuelta

Síntoma reportado: al poner la API en la opción de la interfaz, no se guarda ni se usa.

Lo que está verificado en el código: el modal guarda la clave en `localStorage` bajo `user_gemini_api_key`; el cliente la adjunta como cabecera `x-gemini-api-key` en cada llamada; y el backend la prioriza por sobre la del entorno. La tubería *existe*.

Hipótesis, sin verificar todavía: (a) el camino que realmente corre la app es el de **motores** (catálogo con DeepSeek como principal), y ese camino **sólo consume la clave propia en el proveedor `gemini`** — si el motor activo es otro, la clave guardada nunca se toca; (b) el indicador de la interfaz («activa» / «sin configurar») se inicializa una sola vez al montar y no se refresca después de guardar, así que puede estar mostrando el estado viejo; (c) el modo de inferencia (`local` / `nube` / `auto`) redirige la cadena, y en modo local no hay proveedor que use la clave.

Qué falta: reproducirlo con la consola abierta — mirar si la cabecera sale en la petición (eso separa (a) de (b)) — y después decidir el arreglo. **No está resuelto.**

6.4 · El resto del roadmap, por versión

Versión	Qué trae	Estado
v0.2	Motor de condensación (Fase A hecha: Norte Estratégico + macro-nodo con linaje), runtime local independiente, interruptor Local/Cloud visible, streaming de tokens	Fase A hecho · Fases B y C pendiente
v0.3	Co-agente de voz: dictado, respuesta hablada, conversación por turnos	construido — el idioma de la voz ya está desacoplado de la interfaz y el eco filtrado
v0.4	Control móvil human-in-the-loop por WebSocket, panel para autorizar escrituras	cortado hasta que haya uso real
v0.5	Pipeline de ingesta y síntesis	pendiente
v1.0	Distribución: instaladores firmados + updater, builds macOS/Linux, onboarding desde la UI	Cortado (macOS/Linux) hasta que haya uso real. La firma es lo único vivo de este bloque.

También hay 8 PRs abiertos de Dependabot. Ojo con uno: el bump de TypeScript 5.8 → 7.0.2 **falla la CI**. Los demás (notify, request, tower-http, actions/checkout, setup-node, @types/node) están verdes o sin verificar.

7 · Riesgos y puntos ciegos

Riesgo	Por qué importa
El lienzo no tiene tests de render.	Los 10 tests cubren funciones puras. Arrastrar nodos, conectar aristas y el minimapa se verifican a mano. Después de la migración a la v12 eso es el hueco más grande.
La atención es el recurso escaso.	Seis clientes MCP, cinco apps de agente, tres CLIs: el costo no es la factura, es el ruido. Sin un registro de qué hizo cada worker, no hay decisión sobre cuál mantener.
Permisos de los agentes.	Un worker con escritura en el repo y acceso al llavero es un riesgo real, y ya se materializó una vez. Regla vigente: alcance declarado, worktree propio, revisión por tests antes de mergear.
Las cuotas como variable de diseño.	El grounding de Gemini se agotó y GitHub Models se retiró. Todo rol debe declarar su cuota, su límite y su plan B antes de asignársele trabajo.
16 GB de RAM y una sola GPU.	Con 12 GB de VRAM, dos workers con modelos locales se pisan. Los locales se serializan; los de nube pueden paralelizar.
Bus factor.	Sin <code>AGENTS.md</code> commiteado, sin demo pública y sin instalador firmado, nadie más puede aportar y quedás como único punto de falla. (Dos de esos tres ya están.)
Licencias.	<code>e1kjs</code> es EPL/GPL — descartado para un producto distribuido; <code>dagre</code> es MIT y alcanza. Si NodeFlow se distribuye (MIT), la matriz de licencias de dependencias y modelos hay que mirarla antes de vender, no después.

Lo que decidimos NO hacer ahora

- **Base embebida (redb / SQLite):** el JSON todavía sirve y deja auditar. Primero medir.
- **CRDT (yjs):** es la base de la fase móvil, pero es un proyecto en sí. Va cuando esa fase empiece.
- `e1kjs` : licencia EPL/GPL en un producto que se distribuye.
- **MSIX / Microsoft Store:** cuenta paga y empaquetado, sin usuarios todavía.
- **Migrar a la 2.0-rc de tauri-specta** : es un release candidate; primero verificar compatibilidad con Tauri 2.11.

8 · Los próximos tres movimientos

1. **Cerrar la migración a React Flow 12** — commit, rebase sobre `main` , PR con los árbitros en verde, y prueba manual del lienzo. Media tarde.
2. **Resolver la clave BYOK** — reproducir con la consola abierta, aislar cuál de las tres hipótesis es, y arreglarla. Es lo que hace que la app sirva para que otro la use con su propia cuota.
3. **Los cinco detalles de programa** — bandeja, atajo global, window-state, soltar archivos, paleta de comandos. Una tarde corta, y es lo que más se nota al abrir la app.

Y en paralelo, lo que no es código: **el video demo y los 30 prospectos cierran el 30/09**. El instalador firmado de verdad y la release `v0.3.8` son el soporte material de ese envío.

9 · Fuentes y método

Todo lo marcado como **medido** se verificó el 19/09/2026 contra:

- El repositorio real: `git log`, `gh pr list`, `git worktree list`, `tauri.conf.json`, `package.json`, `Cargo.toml`.
- La suite: `cargo test` y `vitest run` corridos en esta sesión — 282 + 10 en verde.
- Los artefactos: los dos instaladores en `src-tauri/target/release/bundle/`, con `signtool verify`.
- El registro de GitHub: PRs, releases, runs de CI, y los certificados en el almacén de Windows.
- Las fuentes autoritativas para la investigación: `registry.npmjs.org` (versión, fecha, licencia, `peerDependencies`), `crates.io/api/v1` (versión estable, fecha, autoría) y `gh api` (existencia, estrellas, actividad, licencia).

Las afirmaciones de la investigación delegada se conservaron sólo cuando el sondeo las confirmó; las que no se sostienen están listadas con su prueba en el informe de la pila. Lo que está marcado como hipótesis (§6.3) **no** está verificado y se dice explícitamente.

Informe generado para TOMAS.WAV · NodeFlow 0.3.7 · 19 de septiembre de 2026 · rama `main` en `d11f5d3`

Marcas: **medido** (corrido en esta sesión) · **construido** (en `main`, CI verde) · **pendiente** (identificado, con esfuerzo estimado) · **descartado** (evaluado y no conviene ahora).